

App Deck – Technische Dokumentation

Überblick

App Deck ist eine in Java Swing entwickelte Desktop-Anwendung für macOS, die als frei konfigurierbare Schaltflächen-Leiste (à la Stream Deck) fungiert. Sie erlaubt das Öffnen von URLs, Programmen, Dateien und Ordnern sowie das Kopieren von Text in die Zwischenablage über ein Raster von 6x4 Schaltflächen pro Seite mit beliebig vielen Seiten und optionalen Unter-Ordnern.

Systemvoraussetzungen

- Java 21+
- macOS (für .app-Icons via `sips`, App-Erkennung via `osascript` und .app-Bundle-Erstellung via `jpackage`)

Architektur

Paketstruktur

```
src/main/java/streamdeck/  
    StreamDeckApp.java    -- Hauptklasse (GUI, Logik, Icons, Drag & Drop)  
    ButtonConfig.java     -- Datenmodell für eine Schaltfläche  
    ConfigLoader.java     -- JSON-Persistenz (Gson)  
  
src/main/resources/streamdeck/  
    app-icon.png          -- Eingebettetes App-Icon
```

Datenmodell (ButtonConfig)

Jede Schaltfläche wird durch ein ButtonConfig-Objekt repräsentiert:

```
public class ButtonConfig {  
    private String label;           // Anzeigetext  
    private String type;           // URL | PROGRAM | FOLDER | COPY | FILE  
    private String target;         // URL, Pfad, Kommando oder Text  
    private List<List<ButtonConfig>> pages; // nur bei FOLDER: Unterseiten  
    private boolean check;        // YouTube-Update-Prüfung aktiv  
    private String latestVideoId; // zuletzt gesehene Video-ID  
    private boolean hasNew;       // Flag für "neu"-Badge  
}
```

Konfigurationsformat (ConfigLoader)

Die Konfiguration wird als JSON gespeichert, ein Array von Seiten (Arrays von ButtonConfig-Objekten):

```
[  
  [  
    ]  
  ]  
]
```

```

    { "label": "GitHub", "type": "URL", "target": "https://github.com" },
    { "label": "Terminal", "type": "PROGRAM", "target": "open -a Terminal" },
    { "label": "Dokumente", "type": "FILE", "target": "file:///Users/name/Dokumente" },
    { "label": "Passwort", "type": "COPY", "target": "meinPasswort123" }
  ],
  [
    { "label": "Projekt", "type": "FOLDER", "target": "",
      "pages": [
        [
          { "label": "Build", "type": "URL", "target": "https://jenkins.example.com" },
          { "label": "Git", "type": "URL", "target": "https://github.com/mein/projekt" }
        ]
      ]
    }
  ]
]

```

Der `ConfigLoader` unterstützt abwärts kompatible flache JSON-Arrays (einzelnes Array für Seite 0), die automatisch in das Mehrseiten-Format überführt werden.

GUI-Komponenten

Raster

- 6 Spalten x 4 Zeilen = 24 Plätze pro Seite
- Konfigurierbar über Konstanten `COLS` und `ROWS`
- Quadratische Schaltflächen mit 120x120 Pixel
- 14px abgerundete Ecken mit Schlagschatten (Klasse `RoundedShadowBorder`)
- Hellgrauer Gradient (248,248,250 nach 225,225,230) mit bläulichem Hover-Effekt (230,245,255 nach 200,225,240)
- Gedrückter Zustand: dunklerer Gradient (210,210,215 nach 185,185,190)
- Leere Schaltflächen verwenden den gleichen hellen Gradienten wie konfigurierte
- 10px Abstand zwischen den Schaltflächen, 10px Außenabstand
- Text: fett, 12pt, zentriert unter dem Icon
- Hintergrund: dunkler Gradient (35,35,40 nach 50,50,55) via eigenem `JPanel`

Navigationsschaltflächen

Die Navigation erfolgt über zwei spezielle Schaltflächen in der unteren Zeile:

- Letzter Slot (Position `COLS*ROWS-1`): immer vorwärts (Pfeil nach rechts), Tooltip "Nächste Seite"
- Erster Slot der unteren Zeile (Position `BOTTOM_ROW_START`): rückwärts (Pfeil nach links), Tooltip "Vorherige Seite" – nur sichtbar wenn `currentPage > 0` oder ein Ordner geöffnet ist

Versionslabel

Ganz unten: "V1.0 vom 16.05.26" in 9pt, grau (150,150,150).

Funktionen im Detail

Button-Typen

URL: Öffnet die angegebene URL im Standardbrowser (`Desktop.getDesktop().browse()`). Unterstützt `file://`-Pfade für lokale Dateien und Verzeichnisse.

PROGRAM: Startet ein Programm. Akzeptiert: - `open -a AppName` - macOS-App - `open "/Applications/App.app"` - Pfad mit Leerzeichen - `/Pfad/zur/App.app` - Direkter Pfad - Beliebige Shell-Kommandos

FILE: Öffnet eine Datei oder einen Ordner mit der Standard-Anwendung (`Desktop.getDesktop().open()`). Die Auswahl erfolgt über einen Dateiauswahl-dialog mit Tastatur-Navigation.

FOLDER: Erzeugt eine Unterseiten-Struktur. Beim Klick wird in den Ordner navigiert, der eigene Seiten mit bis zu 24 Schaltflächen pro Seite haben kann. Die Rückkehr erfolgt über die Rückwärts-Schaltfläche. Ordnertiefe ist auf eine Ebene beschränkt.

COPY: Kopiert den Zielfeld-Text in die System-Zwischenablage und zeigt kurz "Kopiert!" auf der Schaltfläche an.

Konfigurationsdialog

Der Bearbeiten-Dialog wird als eigener `JDialog` mit `JOptionPane` als Grundlage realisiert:

- Alle Elemente sind linksbündig mit 15px Abstand zum linken Rand
- Eingabefelder dehnen sich bis 15px vor den rechten Rand
- OK und Abbrechen sind zentriert in einer Reihe unterhalb des Formulars
- Der Dialog ist auf dem Bildschirm zentriert, Minimumbreite 520px
- Dialogtitel: "Schaltfläche konfigurieren"

Auto-Erkennung des Typs: Während der Eingabe im Zielfeld wird der Typ automatisch erkannt: - URLs (`http/https`) -> Typ URL, Label-Vorschlag aus Domain - Existierende Dateien/Ordner -> Typ FILE, Label = Dateiname - `file://`-Pfade -> Typ FILE - `open -a ...` -> Typ PROGRAM - Pfade endend auf `.app` -> Typ PROGRAM - Bei YouTube-URLs wird die Checkbox "Auf neue Videos prüfen" automatisch aktiviert und sichtbar geschaltet - Bei anderen URLs bleibt die Checkbox unsichtbar

Label-Vorschlag: Bei URLs wird der Haupt-Domain-Name (ohne TLD) als Label vorgeschlagen. Bei YouTube-URLs wird der @-Handle aus dem Pfad extrahiert (z.B. `@Actuarium` -> `Actuarium`). Bei Programm-Pfaden wird der Dateiname ohne `.app` verwendet.

Browse-Button: Öffnet einen `JFileChooser` mit Tastatur-Navigation (`KeyEventDispatcher` für Type-Ahead). Ein `browseDialogOpen`-Flag verhindert Interferenz mit der globalen Type-to-Search-Funktion.

Rechtsklick-Menü

- **Bearbeiten** – Dialog zum Ändern von Label, Typ und Ziel
- **Ordner anlegen** – Erzeugt einen neuen FOLDER-Button mit einer leeren Seite
- **Entfernen** – Löscht die Schaltfläche (null im Seiten-Array)

Laufende Apps erkennen

Alle 5 Sekunden wird die Liste der laufenden Prozesse via `osascript` und `ps -ef` abgefragt. Programme mit einem laufenden Prozess erhalten einen grünen Rahmen (`ROUNDED_BORDER_RUNNING`, 4px, RGB 0,180,0). Die Erkennung gleicht sowohl den Prozessnamen als auch die Kommandozeile ab, um auch Apps mit abweichenden Namen (z.B. VS Code als 'Code') zu erfassen.

Long-Press zum Beenden

Ein PROGRAM-Button muss >800ms gedrückt gehalten werden, um die App via `osascript` zu beenden. Der grüne Rahmen wird sofort entfernt, und die App erscheint für 12s nicht mehr als laufend (Kühlung, bis der OS-Beendigungsprozess abgeschlossen ist).

Drag & Drop

- Drag-Schwelle: 5px (reagiert schon bei kleinen Bewegungen)
- Ghost-Fenster (`JWindow`) zeigt das Schaltflächen-Bild während des Ziehens
- Hand-Cursor während des Ziehens
- Ablegen auf eine andere Schaltfläche: Tausch der Positionen
- Ablegen auf eine FOLDER-Schaltfläche: Verschieben in den ersten leeren Slot des Ordners
- Ablegen auf die Rückwärts-Schaltfläche (Pfeil links): Verschieben zur vorherigen Seite (oder aus Ordner heraus)
- Ablegen auf die Vorwärts-Schaltfläche (Pfeil rechts): Verschieben zur nächsten Seite (mit `pageIsFull`-Prüfung)
- Long-Press-Timer wird bei Drag-Start abgebrochen
- Bei seitenübergreifendem Drag&Drop wird `ConfigLoader.save()` + `iconCache.clear()` + `updateAllButtons()` aufgerufen

Icon-Ladung

Icons werden asynchron (Hintergrund-Thread) geladen, um die EDT nicht zu blockieren:

1. **Cache** (`ConcurrentHashMap`) – `type` + `":"` + `target` als Key

2. **PROGRAM**: ShellFolder (Reflection) -> FileSystemView -> `sips` für .icns (temp PNG mit Pfad-Hash) -> lokale `favicon.svg` -> System-Icon-Fallback
3. **FILE**: Gleicher Weg wie PROGRAM (`resolveProgramIcon`)
4. **URL**: Google Favicon-Service (<https://www.google.com/s2/favicons?domain=...>) -> lokale `favicon.svg` -> Globe-Fallback
5. **Globe-Fallback** (48x48): Blauer Kreis mit Breiten-/Längengrad-Linien (Java2D)
6. **FOLDER** (48x48): Gelb-oranger Ordner mit Lasche (Java2D)
7. **COPY** (48x48): Blaue Zwischenablage mit Textzeilen (Java2D)
8. Stale-Callback-Guard: Vor dem Setzen des Icons wird geprüft, ob die Schaltfläche noch die gleiche Konfiguration hat

Programmatische Icons (Java2D)

Drei Icons werden ohne externe Dateien zur Laufzeit generiert:

- **Globe**: Blauer Kreis (70,140,220) mit weißen Hilfslinien für Äquator und Nullmeridian
- **Ordner**: Gelb-oranger Gradient (245,215,85 bis 210,175,50) mit braunem Tab und Umriss
- **Kopie**: Hellblaue Briefklammer oben, dunkelblaues Rechteck (100,120,200) mit weißen Textzeilen

YouTube-Update-Prüfung

Die Anwendung kann periodisch YouTube-Kanalseiten auf neue Videos prüfen:

- Aktivierung pro Button über die Checkbox “Auf neue Videos prüfen” (nur sichtbar bei YouTube-URLs)
- Prüffrequenz: konfigurierbar über `CHECK_INTERVAL_MINUTES` (Standard: 5 Minuten)
- Erster Check erfolgt 10 Sekunden nach dem Start
- Erkennung via Regex `"videoId": "[A-Za-z0-9_-]{6,}"` aus dem HTML der Kanal-Seite
- Bei einem neuen Video erscheint ein rotes “neu”-Badge auf dem Button
- Badge wird beim Klick auf den Button zurückgesetzt (`hasNew = false`)
- Bei Ordnern: Der Ordner-Button zeigt ebenfalls ein “neu”-Badge, wenn einer seiner Unter-Buttons ein neues Video hat (dynamisch beim Rendern geprüft)
- Bei erstmaliger Prüfung wird die Video-ID nur gespeichert ohne “neu” zu markieren (Baseline)
- Manueller Trigger über Menüpunkt “Auf neue YouTube-Videos prüfen” mit Warte-Cursor und 3s-Ergebnisdialog
- `latestVideoId` wird nur im JSON gespeichert, wenn sie durch einen Check gesetzt wurde
- Stale/ungültige IDs (kürzer als 6 Zeichen) werden ignoriert

Suchdialog (Cmd+F)

Der Suchdialog durchsucht alle Seiten (inklusive Ordner-Unterseiten) nach Label und Ziel:

- Öffnung über Cmd+F (Strg+F) oder Menüpunkt “Suche”
- Type-to-Search: Bei Tastatureingabe auf nicht-fokussierten `JTextComponent`-Bereichen wird der Suchdialog automatisch geöffnet und das Zeichen vorausgefüllt
- Gefilterte Ergebnisse in einer Liste mit Seitenangabe (z.B. “Actuarium (S.1 > Ordner > S.1)”)
- Navigation in der Ergebnisliste mit Pfeiltasten (oben/unten)
- Enter wählt das markierte Ergebnis aus
- Doppelklick wählt das Ergebnis aus
- Escape schließt den Dialog
- `searchDialogOpen`-Flag verhindert rekursive `KeyEventDispatcher`-Aufrufe

Navigation zum Ergebnis: Nach Auswahl eines Ergebnisses wird zur entsprechenden Seite und Position navigiert. Der gefundene Button erhält für 5 Sekunden einen blinkenden gelb-orangen Rahmen (400ms Intervall). Der Button wird fokussiert.

Tastatur-Navigation

- **Pfeiltasten:** Navigation zwischen den Raster-Buttons (oben/unten/links/rechts)
- **Enter:** Führt die Aktion des fokussierten Buttons aus (`doClick()`)
- **ESC:** Rückwärts-Navigation (`prevPage()` oder `leaveFolder()`)
- **Fokus-Ring:** Gold-oranger gestrichelter Rahmen (4px, 8/6 Dash-Muster) wird in `paint()` nach `super.paint(g)` gezeichnet, so dass er oberhalb des grünen Running-Rahmens sichtbar ist

Menüleiste

- **App Desk:** Über-App-Desk, Trennlinie, “Suche mit Strg + F”, Trennlinie, “Auf neue YouTube-Videos prüfen”, Trennlinie, “App Desk beenden”
- **Hilfe:** “Dokumentation anzeigen” (öffnet `Dokumentation.pdf` via `Desktop.open()`)
- Hilfe ist rechtsbündig, App Desk linksbündig

macOS .app-Bundle

Das Skript `build-app.sh` erzeugt ein .app-Bundle via `jpackage`:

```
./build-app.sh  
open "App Deck.app"
```

Eigenschaften: - Bundle-ID: `com.dobronski.appdeck` - Custom `.icns`-Icon (farbiges Raster auf dunklem Hintergrund) - Dock-Name via `System.setProperty("apple.awt.application.name", "App Deck")`

"App Deck") - Dock-Icon via `Taskbar.setTaskbarIcon()`

Logging

Die Anwendung schreibt Logs in `appdeck.log` im gleichen Verzeichnis wie `config.json`:

- Format: `yyyy.MM.dd_HH:mm:ss` + Meldung
- Rotation bei 5 MB (`appdeck.log` -> `appdeck.1.log` -> ... -> `appdeck.9.log`)
- Log-Einträge: Start/Ende, YouTube-Prüfung (Start, pro Button, Ergebnis, nächster Check), Fehler

Build und Ausführung

JAR (einfach)

```
mvn package -q
java -jar target/streamdeck-1.0-SNAPSHOT.jar [config.json]
```

macOS .app Bundle

```
./build-app.sh
open "App Deck.app"
```

Konfigurationspfad

1. Kommandozeilen-Argument, falls angegeben
2. `config.json` im aktuellen Verzeichnis
3. `~/Library/Application Support/App Deck/config.json` (wird automatisch angelegt)

Abhängigkeiten

- **Gson 2.11.0** – JSON-Serialisierung (`com.google.code.gson:gson`)
- Keine weiteren externen Bibliotheken (reines Java Swing + macOS-native Tools)

Wichtige Konstanten

Konstante	Wert	Beschreibung
COLS	6	Spalten im Raster
ROWS	4	Zeilen im Raster
BUTTON_SIZE	120	Seitenlänge der quadratischen Buttons (px)

Konstante	Wert	Beschreibung
BOTTOM_ROW_START	$(ROWS-1)*COLS$	Index der ersten Schaltfläche in der unteren Zeile
ICON_SIZE	48	Zielgröße der Icons (px)
DRAG_THRESHOLD	5	Pixel für Drag-Erkennung
ARC	14	Rundungsradius der Buttons
SHADOW	3	Schlagschatten-Versatz
CHECK_INTERVAL_MINUTES	5	Minuten zwischen YouTube-Prüfungen

Index-Berechnungen

Die Navigationselemente belegen Slots in der unteren Zeile:

- Letzter Slot ($COLS*ROWS-1$): immer Vorwärts (Pfeil rechts)
- Erster Slot der unteren Zeile (BOTTOM_ROW_START): Rückwärts (Pfeil links) wenn Seite > 0 oder Ordner geöffnet

`gridToPageIndex()` und `pageToGridIndex()` bilden zwischen Gitter-Position und Seiten-Index ab, wobei Navigations-Slots übersprungen werden. `findEmptySlot()` und `pageIsFull()` sind Hilfsmethoden für Drag & Drop und Ordner-Befüllung.

Running-App-Erkennung

```
isAppRunning(appKey) = runningApps.contains(appKey)
|| runningCmdLines.contains(appKey)
```

- `runningApps`: via `osascript` ermittelte Prozessnamen
- `runningCmdLines`: via `ps -ef` ermittelte Kommandozeilen
- `killedApps`: via Long-Press beendete Apps (12s Kühlung)

Long-Press-Timing

Phase	Dauer	Aktion
Gedrückt halten	>800ms	<code>osascript</code> quit, Rahmen entfernen, App in <code>killedApps</code>

Phase	Dauer	Aktion
Kühlung	12s	App erscheint nicht als laufend, damit der Beendigungsprozess abgeschlossen werden kann

Beispielkonfiguration

Eine vollständige Konfiguration mit zwei Seiten und einem Ordner:

```
[
  [
    { "label": "GitHub", "type": "URL", "target": "https://github.com" },
    { "label": "Terminal", "type": "PROGRAM", "target": "open -a Terminal" },
    { "label": "Projekte", "type": "FOLDER", "target": "", "pages": [
      [
        { "label": "Build", "type": "URL", "target": "https://jenkins.example.com" },
        { "label": "Wiki", "type": "URL", "target": "https://wiki.example.com" }
      ]
    ]
  ]
],
[
  { "label": "Notizen", "type": "PROGRAM", "target": "open -a Notes" },
  { "label": "Codeschnipsel", "type": "COPY", "target": "System.out.println(\"Hello World\" )"},
  { "label": "Actuarium", "type": "URL", "target": "https://www.youtube.com/@Actuarium/vi"
}
]
```

Icon-Generierung

Das App-Icon (farbiges Raster auf dunklem Hintergrund) wird durch `icons/make-icon.sh` generiert, das ein temporäres Java-Programm kompiliert und ausführt, um ein 1024x1024 PNG zu erzeugen, das dann via `sips` und `iconutil` in ein `.icns`-Format gebracht wird.

```
./icons/make-icon.sh
```

Erzeugt: - `icons/app-icon.png` (1024x1024) - `icons/app-icon.icns` (macOS Icon-Format)